Notions théoriques de classe

Tous droits réservés à STUDI - Reproduction interdite

Table des matières

I. Contexte **3**

II. Les classes et objets **3**

A. Les classes 3

B. Les objets 4

C. Exercice : Quiz 8

III. L'encapsulation **10**

A. Introduction 10

B. En quoi ça consiste ? 10

C. Accesseur et mutateur 11

D. Exercice : Quiz 13

IV. Essentiel **15**

V. Auto-évaluation **16**

A. Exercice 16

B. Test 16

Solutions des exercices **18**

Tous droits réservés à STUDI - Reproduction interdite

I. Contexte

Durée : 60 minutes

Matériel : IDE

Contexte

La Programmation Orientée Objet (POO) est un style de programmation qui permet de structurer le code autour d'objets du monde réel. Chaque objet peut être comparé à un objet du quotidien : une voiture, un téléphone, ou une maison. Ces objets ont des caractéristiques (appelées *attributs*) et des actions qu'ils peuvent réaliser (appelées *méthodes*). Par exemple, une voiture a des caractéristiques comme la couleur, le modèle, ou le type de carburant, et peut effectuer des actions comme démarrer, freiner ou klaxonner.

Le concept de la POO est très utile pour modéliser des systèmes complexes de manière intuitive et claire. Nous allons voir comment, avec Python, on peut représenter les objets du monde réel sous forme de classes et comment créer des instances de ces classes pour manipuler des objets. La classe sert de plan de construction, un peu comme une recette de cuisine, et chaque objet en est une réalisation concrète, comme un plat préparé à partir de la recette.

II. Les classes et objets

A. Les classes

Définition **Classe**

Une classe peut être comparée à un plan, une sorte de moule à partir duquel on peut créer plusieurs objets similaires. Imaginez une classe comme une recette de cuisine. La recette contient les instructions et les ingrédients nécessaires, mais elle n'est pas le plat lui-même. De la même façon, une classe en Python contient des définitions (méthodes) et des attributs, mais elle n'est pas encore un « *objet réel* » que l'on peut utiliser.

Pensez à une classe comme le plan d'une maison. Le plan indique comment construire la maison (où placer les murs, les fenêtres, etc.), mais il n’est pas une maison physique. Ce plan peut être utilisé pour construire plusieurs maisons (des objets), qui seront toutes basées sur la même structure mais peuvent avoir des détails différents (couleurs, tailles, etc.).

En Python, on définit une classe avec le mot-clé `class` :

```
1 class Maison:
2     pass  # Pour l'instant, la classe ne contient rien
```

Ici, nous avons défini une classe appelée *Maison*. Cependant, elle ne fait encore rien. Elle est comme un plan vide. Nous avons besoins des attributs afin de personnaliser notre maison, les attributs sont des variables qui vont contenir les valeurs des caractéristiques de notre maison.

B. Les objets

Définition **Les attributs**

Les **attributs** sont des variables associées à une instance de classe, c'est-à-dire à un objet. Ces attributs représentent les caractéristiques spécifiques de chaque objet. Par exemple, si l'on prend une maison, elle pourrait avoir des attributs comme :

- **Couleur** (ex. : rouge, blanche, bleue)
- **Nombre de pièces** (ex. : 4, 5, 6)
- **Superficie** (ex. : 100, 120, 150 m^2)

Exemple

Si vous avez deux maisons, l'une peut être rouge avec 5 pièces, et l'autre blanche avec 4 pièces. Chaque maison a ses propres caractéristiques, bien qu'elles soient toutes deux des maisons.

Ajout d'attributs à la classe *Maison* :

```
1 class Maison:
2     def __init__(self, couleur, nombre_de_pieces, superficie):
3         self.couleur = couleur
4         self.nombre_de_pieces = nombre_de_pieces
5         self.superficie = superficie
```

Dans cet exemple :

- couleur, nombre_de_pieces, et superficie sont des **attributs**.
- Ces attributs sont initialisés à l'aide du **constructeur** __init__ lors de la création d'un objet.

Définition **Les constructeurs**

Nous avons utilisé '__init__' , il s'agit d'un constructeur. Un **constructeur** est une méthode spéciale qui est appelée automatiquement lorsque vous créez un nouvel objet à partir d'une classe. En Python, le constructeur est la méthode __init__. Il permet d'**initialiser les attributs** de l'objet, c'est-à-dire de leur donner des valeurs initiales.

Le constructeur est le fameux plan, qui va donner la démarche afin d'initialiser les attributs et de concevoir notre objet. Un **objet** est une instance d'une classe. Si une classe est un plan ou un moule, un objet est une version concrète, un produit fini construit à partir de ce plan. Chaque objet possède ses propres valeurs pour les attributs définis par la classe.

Exemple

Voyons ensemble un exemple plus concret, nous avons dans notre maison un constructeur, on sait que nous avons besoin pour construire une maison, de plusieurs informations, à savoir une couleur, un nombre de pièces ainsi qu'une superficie.

Nous pouvons construire une ou plusieurs maisons à partir de ce même constructeur. Chaque maison sera un objet unique avec ses propres valeurs d'attribut.

```
1 ma_maison = Maison(couleur='bleu',nombre_de_pieces=6,superficie=123)
2 ma_maison_deux = Maison(couleur='rouge',nombre_de_pieces=7,superficie=256)
```

ma_maison ainsi que *ma_maison_deux* sont des instances de *Maison*.

Tous droits réservés à STUDI - Reproduction interdite

Self

Dans notre constructeur, nous avons utilisé « *self* » qui permet de faire reference à l'objet actuel. Dans une méthode de classe ou un constructeur, self permet d'accéder aux attributs et aux autres méthodes de classe. Sans self, Python ne saurait pas quel objet il doit manipuler.

Méthode

Par exemple, dans la méthode __init__ que nous avons aperçue ensemble, *self*, fait référence à l'instance de la classe qui est en train d'être créée. Cela permet à l'objet de stocker ses attributs individuels.

```
1 class Maison:
2     def __init__(self, couleur, nombre_de_pieces, superficie):
3         self.couleur = couleur  # self fait référence à l'instance de la classe Maison
4         self.nombre_de_pieces = nombre_de_pieces
5         self.superficie = superficie
```

Une fois que nous avons créé notre objet, nous allons avoir besoin de, par exemple, afficher les informations à propos de notre maison. Pour cela, nous avons besoin des méthodes de classe. Une **méthode de classe** est une fonction définie au sein d'une classe qui permet à notre objet d'effectuer des actions.

Par exemple, nous pouvons avoir besoin d'afficher les informations de notre maison :

```
1 class Maison:
2     def __init__(self, couleur, nombre_de_pieces, superficie):
3         self.couleur = couleur
4         self.nombre_de_pieces = nombre_de_pieces
5         self.superficie = superficie
6
7     def afficher_info(self):
8         print(f"Maison {self.couleur} avec {self.nombre_de_pieces} pièces, "
9               f"surface totale de {self.superficie} m².")
```

Remarque

Si on essaye dans *afficher_info*, d'essayer via self d'accéder à un attribut n'existant pas, comme par exemple self.promoteur, Python nous renverra une erreur *AttributeError*

Méthode

Essayons de concevoir notre objet maintenant et d'appeler la méthode :

```
1 ma_maison = Maison(couleur="Rouge",nombre_de_pieces=6,superficie=150)
2 ma_maison.afficher_info()  # Appel de la méthode afficher_info sur l'objet ma_maison
3 # Affiche : Maison Rouge avec 6 pièces, surface totale de 150 m².
```

Avec Python 3.12, l'analyse syntaxique a été améliorée, rendant la gestion des types plus efficace et plus performante.

Les **annotations de type** permettent d'indiquer le type de données attendu pour les attributs et les arguments des méthodes. Cela ne change pas le comportement du programme mais permet de rendre le code plus lisible et aide les outils de développement à détecter des erreurs plus facilement. Ainsi, nous pouvons dire par exemple à notre code que 'couleur' est forcément une chaine de caractère.

```
1 class Maison:
2     def __init__(self, couleur: str, nombre_de_pieces: int, superficie: int):
3         self.couleur = couleur
4         self.nombre_de_pieces = nombre_de_pieces
5         self.superficie = superficie
```

Tous droits réservés à STUDI - Reproduction interdite

Cas pratique : Gérer une collection de livres

Afin de nous entrainer un peu, voyons un cas pratique pour mettre en place toutes les notions ! Notre ami, José, a besoin de nous car il a ouvert récemment une bibliothèque et souhaite gérer efficacement sa collection de livres. Il a besoin d'un système simple pour ajouter des livres, afficher les livres disponibles et éviter d'ajouter plusieurs fois le même livre.

Premièrement, José nous précise qu'un livre possède un titre, un auteur ainsi qu'un nombre de page. Nous pouvons créer dans un premier temps, une class *Livre*

```
1 class Livre:
2     def __init__(self, titre, auteur, pages):
3         # Initialisation des attributs du livre
4         self.titre = titre
5         self.auteur = auteur
6         self.pages = pages
```

Explication :

- Le constructeur __init__ est utilisé pour initialiser les attributs titre, auteur, et pages de chaque livre.
- Lorsque José ajoutera un livre, il devra fournir ces trois informations (de manière obligatoire), qui seront stockées dans un objet Livre.

Dans un second temps, José souhaite pouvoir afficher les informations d'un livre, nous devons ainsi proposer un affichage avec le titre, l'auteur ainsi que le nombre de pages.

```
1 class Livre:
2     def __init__(self, titre, auteur, pages):
3         self.titre = titre
4         self.auteur = auteur
5         self.pages = pages
6
7     def afficher_info(self):
8         # Affichage des informations du livre
9         print(f"Livre : {self.titre}, écrit par {self.auteur}, {self.pages} pages.")
```

Explication :

- La méthode afficher_info permet d'afficher les détails du livre.
- Elle utilise les attributs titre, auteur, et pages pour fournir ces informations.

Maintenant que nous avons modélisé un livre, nous devons essayer d'en faire une liste, car une bibliothèque contient plusieurs livres, et non un seul. Pour cela, créons une classe *Bibliothèque* qui contient une liste de *Livre*.

```
1 from typing import List
2 class Bibliotheque:
3     def __init__(self):
4         # Initialisation d'une liste vide pour stocker les livres
5         self.livres : List[Livre] = []
```

Explication :

Le constructeur __init__ de la classe Bibliotheque initialise une liste vide de livres qui servira à stocker les objets *Livre*.

José nous demande ensuite de pouvoir ajouter des livres à sa liste de livres existante. Les listes en Python detiennent une fonction 'append()' qui permet d'ajouter un élément. Nous allons devoir créer une méthode de classe dans *Bibliothèque* afin d'ajouter ce livre à notre liste.

```
1 from typing import List
2 class Bibliotheque:
3     def __init__(self):
4         # Initialisation d'une liste vide pour stocker les livres
5         self.livres : List[Livre] = []
```


Tous droits réservés à STUDI - Reproduction interdite

```
    def ajouter_livre(self, livre):
        # Ajout d'un livre à la bibliothèque
        self.livres.append(livre)
        print(f"Livre '{livre.titre}' ajouté à la bibliothèque.")
```

Explication :

- La méthode ajouter_livre prend un objet Livre en paramètre et l'ajoute à la liste livres.
- Un message de confirmation est affiché pour informer que le livre a été ajouté.

José, nous indique que notre développement lui convient, mais il veut qu'on l'améliore pour éviter les doublons. Un livre est caractérisé par un auteur et un titre, dès lors, si notre liste détient un livre qui contient le même auteur et titre que le livre à ajouter, nous pouvons supposer qu'il s'agit de la même chose. Modifions de cette façon notre méthode de classe *ajouter_livre*.

```
from typing import List
class Bibliotheque:
    def __init__(self):
        # Initialisation d'une liste vide pour stocker les livres
        self.livres : List[Livre] = []

    def ajouter_livre(self, livre):
        # Vérification de doublon : on vérifie si le livre est déjà dans la bibliothèque
        for l in self.livres:
            if l.titre == livre.titre and l.auteur == livre.auteur:
                print(f"Le livre '{livre.titre}' de {livre.auteur} est déjà dans la bibliothèque.")
                return
        # Si le livre n'est pas un doublon, on l'ajoute
        self.livres.append(livre)
        print(f"Livre '{livre.titre}' ajouté à la bibliothèque.")
```

Explication :

- Avant d'ajouter un livre, la méthode vérifie si un livre avec le même titre et auteur existe déjà dans la bibliothèque.
- Si un doublon est trouvé, elle n'ajoute pas le livre et affiche un message d'avertissement.

Enfin, José souhaite que nous puissons lui afficher tous les détails des livres présents dans sa bibliothèque.

Pour ce faire, nous devons parcourir notre liste de livre et afficher les informations à chaque fois, créons la méthode de classe « *afficher_livres* ».

```
from typing import List
class Bibliotheque:
    def __init__(self):
        # Initialisation d'une liste vide pour stocker les livres
        self.livres : List[Livre] = []

    def ajouter_livre(self, livre):
        # Vérification de doublon : on vérifie si le livre est déjà dans la bibliothèque
        for l in self.livres:
            if l.titre == livre.titre and l.auteur == livre.auteur:
                print(f"Le livre '{livre.titre}' de {livre.auteur} est déjà dans la bibliothèque.")
                return
        # Si le livre n'est pas un doublon, on l'ajoute
        self.livres.append(livre)
        print(f"Livre '{livre.titre}' ajouté à la bibliothèque.")

```

Tous droits réservés à STUDI - Reproduction interdite

```
    def afficher_livres(self):
        # Affichage de tous les livres de la bibliothèque
        if self.livres:
            print("Livres dans la bibliothèque :")
            for livre in self.livres:
                livre.afficher_info()
        else:
            print("La bibliothèque est vide.")
```

C. Exercice : Quiz

[solution n°1 p.19]

Question 1

Qu'est-ce qu'une classe en Python ?

- Un type de boucle
- Un plan pour créer des objets
- Une fonction spéciale
- Une variable globale

Question 2

Quel est le rôle du constructeur __init__ dans une classe ?

- Il initialise les attributs de l'objet
- Il supprime l'objet
- Il ajoute l'objet à une liste
- Il modifie une méthode

Question 3

Que représente self dans une méthode de classe ?

- Une référence à la classe
- Une référence à l'objet actuel
- Une variable globale
- Un type de boucle

Question 4

Quelle est la syntaxe correcte pour définir une classe en Python ?

- class NomClasse {}
- class NomClasse:
- def class NomClasse:
- NomClasse = class {}

Question 5

Tous droits réservés à STUDI - Reproduction interdite

Que signifie instancier une classe ?

- Supprimer une classe
- Créer un objet à partir d'une classe
- Modifier un attribut de la classe
- Exécuter une méthode de la classe

Question 6

Que se passe-t-il si vous ne définissez pas de constructeur dans une classe ?

- Python ne pourra pas créer d'objets
- Python créera un constructeur par défaut
- L'objet aura des valeurs aléatoires
- Python lancera une erreur

Question 7

Comment ajouter un attribut à un objet dans une classe ?

- Utiliser self.attribut
- Utiliser class.attribut
- Utiliser def attribut
- Utiliser attr.attribut

Question 8

Comment accéder à un attribut d'instance à l'intérieur d'une méthode de classe ?

- En utilisant class.attribut
- En utilisant self.attribut
- En utilisant attribut.instance
- En utilisant __init__(attribut)

Question 9

Que renvoie la fonction type() lorsqu'elle est appliquée à un objet ?

- Le nom de la méthode
- Le type de l'objet (la classe à laquelle il appartient)
- La taille de l'objet
- La valeur des attributs de l'objet

Question 10

Tous droits réservés à STUDI - Reproduction interdite

Que se passe-t-il si vous tentez d'accéder à un attribut inexistant d'un objet ?

- Python renverra None
- Python lancera une exception AttributeError
- Python créera automatiquement cet attribut
- Python renverra une valeur par défaut

III. L'encapsulation

A. Introduction

L'encapsulation en Python est une pratique essentielle pour sécuriser et protéger les attributs d'une classe. Cela permet de gérer l'accès aux attributs de manière plus contrôlée, en utilisant des getter et setter.

B. En quoi ça consiste ?

Définition **Encapsulation**

L'encapsulation est un principe clé de la programmation orientée objet. Elle consiste à restreindre l'accès direct aux attributs d'un objet depuis l'extérieur de la classe. En encapsulant les attributs, on les protège des modifications accidentelles ou non souhaitées.

Pensons à une voiture. Nous pouvons interagir avec certains éléments de la voiture, comme le volant ou les pédales, mais on ne peut pas accéder directement au moteur. Le moteur est "encapsulé", et nous devons utiliser les contrôles appropriés pour le manipuler.

Exemple

En Python, on peut indiquer qu'un attribut est protégé (non destiné à être modifié directement) en le préfixant d'un seul underscore (`_`). Ce n'est qu'une convention : cela signifie que l'attribut peut être modifié, mais qu'il ne devrait pas l'être directement. Le développeur est encouragé à utiliser des méthodes pour accéder et modifier ces attributs.

Voyons un exemple :

```python
class Voiture:
    def __init__(self, marque, vitesse_max):
        self.marque = marque  # Attribut public
        self._vitesse_max = vitesse_max  # Attribut protégé

voiture = Voiture("Tesla", 250)
print(voiture.marque)  # Accessible directement : Tesla
print(voiture._vitesse_max)  # Accessible mais déconseillé : 250
```

Explication :

- `marque` est un attribut public, accessible et modifiable librement.
- `_vitesse_max` est un attribut protégé. Bien qu'il soit techniquement accessible de l'extérieur, la convention `_` indique qu'il ne doit pas être manipulé directement.

Si on souhaite rendre un attribut véritablement privé, on le préfixe de deux underscores (`__`). Cela déclenche un mécanisme interne appelé le 'name mangling', qui modifie le nom de l'attribut afin qu'il soit moins accessible en dehors de la classe.

Tous droits réservés à STUDI - Reproduction interdite

Modifions notre exemple de voiture :

```
1 class Voiture:
2     def __init__(self, marque, vitesse_max):
3         self.marque = marque
4         self.__vitesse_max = vitesse_max  # Attribut privé
5
6 voiture = Voiture("Tesla", 250)
7 print(voiture.marque)  # Accessible : Tesla
8 print(voiture.__vitesse_max)  # Erreur ! Attribut inaccessible
```

'__vitesse_max' est un attribut privé. Il n'est pas accessible directement depuis l'extérieur de la classe. Si on essaye de l'imprimer ou de le modifier directement, Python lèvera une erreur.

```
Traceback (most recent call last):
  File "/Users/Shared/pythonProject/main.py", line 8, in <module>
    print(voiture.__vitesse_max)  # Erreur ! Attribut inaccessible
          ^^^^^^^^^^^^^^^^^^^^^^
AttributeError: 'Voiture' object has no attribute '__vitesse_max'
```

Tentative d'accès à un attribut privé

Remarque

En interne, Python renomme l'attribut pour éviter tout accès direct. Par exemple, l'attribut `__vitesse_max` sera renommé en `_Voiture__vitesse_max`. Bien que cet attribut soit techniquement accessible, son accès direct est fortement déconseillé.

C. Accesseur et mutateur

L'encapsulation permet de protéger les attributs, mais comment alors les rendre accessibles de manière contrôlée ? C'est là qu'intervient le mécanisme des propriétés. En Python, les propriétés permettent de définir des getter et des setter pour accéder et modifier les attributs en toute sécurité, tout en gardant un contrôle total sur leur modification.

Reprenons l'exemple de la voiture. Même si nous n'avons pas un accès direct au moteur, nous pouvons contrôler la vitesse à l'aide de la pédale d'accélération. De même, les propriétés permettent de manipuler des attributs tout en s'assurant qu'ils sont gérés correctement.

Exemple

En python, nous utilisons le décorateur 'property' afin de pouvoir accéder et modifier les attributs que nous aurons protégé via l'encapsulation.

Voyons dans un premier cas, comment accéder à un attribut protégé.

```
1 class Voiture:
2     def __init__(self, marque, vitesse_max):
3         self.marque = marque
4         self._vitesse_max = vitesse_max  # Attribut protégé
5
6     @property
7     def vitesse_max(self):
8         # Getter pour accéder à l'attribut protégé
```

Tous droits réservés à STUDI - Reproduction interdite

```
        return self._vitesse_max

voiture = Voiture("Tesla", 250)
print(voiture.vitesse_max)  # Appelle automatiquement la méthode vitesse_max (getter) : 250
```

Explication :

- Le *getter* permet d'accéder à l'attribut `*_vitesse_max*` en toute sécurité via la méthode `vitesse_max`. Depuis l'extérieur, l'attribut est accessible comme une variable normale, mais en réalité, la méthode est appelée.
- `*@property*` transforme la méthode en un attribut accessible, tout en encapsulant la logique derrière cet accès.

Ensuite, le décorateur `@property` peut également être utilisé pour définir des setters. Un setter est une méthode qui contrôle comment un attribut peut être modifié. Cela permet d'ajouter des règles ou des validations lorsque l'attribut est mis à jour.

```
class Voiture:
    def __init__(self, marque, vitesse_max):
        self.marque = marque
        self._vitesse_max = vitesse_max

    @property
    def vitesse_max(self):
        return self._vitesse_max

    @vitesse_max.setter
    def vitesse_max(self, valeur):
        # Validation lors de la modification de la vitesse
        if valeur < 0:
            raise ValueError("La vitesse ne peut pas être négative.")
        self._vitesse_max = valeur

voiture = Voiture("Tesla", 250)
voiture.vitesse_max = 300  # Modifie l'attribut via le setter
print(voiture.vitesse_max)  # 300
```

Dans ce cas, le setter permet de définir des règles lors de la modification de l'attribut. Dans cet exemple, la vitesse ne peut pas être négative. De plus, lorsque nous essayons de modifier `vitesse_max`, Python appelle automatiquement la méthode `vitesse_max.setter`, où la validation est effectuée.

On peut légitimement se poser la question « *à quoi cela sert concrètement ?* », car à part les getter et setter, il n'y a aucune différence avec des attributs public.

Les différences fondamentales, sont les choses suivantes :

- Sécurité : nous pouvons contrôler comment un attribut est lu et modifié.
- Validation : nous pouvons ajouter des vérifications lorsque des valeurs sont modifiées (comme dans l'exemple précédent avec la vitesse négative).
- Encapsulation : on encapsule la logique d'accès à un attribut, ce qui permet de garder le contrôle total sur sa manipulation.

Cas pratique : Gérer la confidentialité des salaires

Le mieux pour s'apercevoir du gain avec l'encapsulation, voyons un cas concret.

José souhaite gérer les salaires de ses employés de manière sécurisée. Il veut s'assurer que les salaires ne puissent pas être définis en dessous d'un certain montant minimum.

Tous droits réservés à STUDI - Reproduction interdite

Tout d'abord, nous allons définir une class *Employe*. Le nom sera en public car tout le monde peut savoir le nom d'un employé, mais nous allons protéger son salaire via l'encapsulation.

```
class Employe:
    def __init__(self, nom, salaire):
        self.nom = nom
        self._salaire = salaire

    @property
    def salaire(self):
        return self._salaire

    @salaire.setter
    def salaire(self, montant):
        if montant < 1200:
            raise ValueError("Le salaire ne peut pas être inférieur à 1200€.")
        self._salaire = montant

    def afficher_info(self):
        print(f"Employé: {self.nom}, Salaire: {self.salaire}€")
```

Le salaire étant protégé, nous avons défini un getter et un setter qui modifie le salaire seulement si le montant est supérieur à 1 200.

Nous pouvons ensuite utiliser notre class dans notre programme Python :

```
employe = Employe("José", 1500)
employe.afficher_info()  # Affiche : Employé: José, Salaire: 1500€

employe.salaire = 2000  # Modification sécurisée du salaire
employe.afficher_info()  # Affiche : Employé: José, Salaire: 2000€

# Tentative de définir un salaire trop bas
employe.salaire = 1000  # Erreur : Le salaire ne peut pas être inférieur à 1200€
```

D. Exercice : Quiz

[solution n°2 p.21]

Question 1

Qu'est-ce que l'encapsulation en Python ?

- ○ La création d'objets
- ○ La protection des attributs d'une classe
- ○ La suppression des méthodes inutiles
- ○ L'héritage de classes

Question 2

Que signifie le préfixe d'un attribut avec un seul underscore (_attribut) en Python ?

- ○ L'attribut est public
- ○ L'attribut est privé et inaccessible
- ○ L'attribut est protégé, mais accessible depuis l'extérieur de la classe
- ○ L'attribut est accessible uniquement dans les méthodes statiques

Question 3

Tous droits réservés à STUDI - Reproduction interdite

Que signifie le double underscore (__attribut) en Python ?

- Il cache l'attribut pour qu'il ne soit pas accessible directement depuis l'extérieur
- Il empêche l'attribut d'être modifié même depuis l'intérieur de la classe
- Il crée un attribut partagé entre toutes les instances
- Il rend l'attribut accessible uniquement dans une méthode spécifique

Question 4

Comment accède-t-on à un attribut privé défini avec double underscore (__attribut) ?

- En utilisant self.__attribut
- En utilisant _NomClasse__attribut
- En utilisant @property
- Il est impossible d'accéder à un attribut privé

Question 5

Quel est l'avantage principal du décorateur @property ?

- Il supprime un attribut
- Il permet de définir un getter pour un attribut de manière sécurisée
- Il permet d'accéder à des méthodes privées
- Il empêche l'exécution de méthodes

Question 6

Que fait un setter dans une propriété Python ?

- Il permet d'accéder à un attribut protégé
- Il empêche la modification d'un attribut
- Il permet de définir des règles lors de la modification d'un attribut
- Il renvoie une erreur si l'attribut est modifié

Question 7

Quelle est l'utilité du self dans les getters et setters ?

- Il empêche l'accès à l'attribut
- Il fait référence à l'objet courant, permettant d'accéder aux attributs de l'instance
- Il modifie le type de l'attribut
- Il remplace un attribut public par un privé

Question 8

Tous droits réservés à STUDI - Reproduction interdite

Pourquoi encapsuler les attributs en Python ?

- Pour empêcher les utilisateurs de la classe d'accéder aux méthodes
- Pour sécuriser les données et les protéger d'un accès ou d'une modification non autorisés
- Pour rendre les attributs inaccessibles même dans la classe
- Pour éviter les erreurs de syntaxe

Question 9

Comment empêche-t-on la modification directe d'un attribut tout en permettant l'accès contrôlé ?

- En définissant l'attribut avec un double underscore
- En utilisant une méthode publique pour modifier l'attribut
- En utilisant @property pour accéder à l'attribut et un setter pour le modifier
- En utilisant __del__

Question 10

Quelle est la convention pour indiquer qu'un attribut est « *protégé* » mais pas totalement privé ?

- Utiliser self.attribut
- Utiliser self.__attribut
- Utiliser self._attribut
- Utiliser self.attribut_protected

IV. Essentiel

En Python, une classe est un modèle ou un plan qui sert à créer des objets. Elle définit à la fois les attributs (les caractéristiques) et les méthodes (les actions) que ces objets peuvent avoir. Une classe peut être comparée à une recette, tandis que l'objet est le résultat concret de cette recette. L'objet est donc une instance particulière de la classe, qui possède ses propres valeurs pour chaque attribut.

Lorsqu'un objet est créé, il est initialisé grâce à une méthode spéciale appelée constructeur, définie par `__init__`. Cette méthode est utilisée pour attribuer des valeurs initiales aux attributs de l'objet dès sa création. Sans elle, un objet pourrait être créé sans que ses caractéristiques essentielles ne soient définies.

Les méthodes sont les actions que les objets peuvent exécuter. Ce sont des fonctions définies au sein d'une classe et qui agissent sur les attributs des objets. Elles sont indispensables pour permettre à l'objet de réagir à des événements ou de modifier ses caractéristiques.

L'encapsulation est un des principes fondamentaux de la programmation orientée objet. Elle consiste à protéger les données sensibles des objets pour empêcher leur modification directe depuis l'extérieur de la classe. L'objectif est de rendre certains attributs inaccessibles directement et de ne les modifier que via des méthodes dédiées. Cela assure la cohérence des données au sein de l'objet.

En Python, on utilise des conventions pour marquer un attribut comme « *protégé* » ou « *privé* ». Un seul underscore (`_attribut`) indique qu'un attribut est protégé et qu'il ne devrait pas être modifié directement. Il reste cependant accessible. Un double underscore (`__attribut`), en revanche, rend un attribut privé en le rendant inaccessible directement depuis l'extérieur de la classe.

Tous droits réservés à STUDI - Reproduction interdite

Pour accéder ou modifier ces attributs de manière contrôlée, Python propose un mécanisme efficace : les propriétés. Grâce au décorateur `@property`, il est possible de transformer des méthodes en attributs accessibles de manière simple et intuitive, tout en gardant un contrôle total sur leur accès. Cela permet d'utiliser des getters et setters pour lire et écrire des valeurs dans les attributs de manière sécurisée.

V. Auto-évaluation

A. Exercice

José possède un magasin de vente de produits électroniques et souhaite développer un système de gestion de son inventaire. Ce système doit permettre de suivre les produits, de gérer les prix et les quantités en stock, tout en s'assurant que les prix ne puissent pas être négatifs et que la quantité en stock ne puisse pas dépasser une certaine limite.

Question 1

[solution n°3 p.24]

Concevoir la classe Produit qui dispose d'un 1 attribut public, le nom. Puis, de deux attributs prix et quantite_stock en privée.

Question 2

[solution n°4 p.24]

José souhaite pouvoir accéder au prix et le modifier mais avec une valeur toujours supérieur à 0.

Question 3

[solution n°5 p.24]

José souhaite également accéder à la quantite_stock et de le modifier, mais cette fois ci, la valeur doit forcement être inférieur à 1 000.

Question 4

[solution n°6 p.24]

José souhaite maintenant gérer une liste de produit et il veut que nous fassions une méthode afin de calculer la somme total de tous les produits. Nous allons aborder cela par étape, premièrement, nous allons créer une class Stock qui va contenir une liste de produit. Ensuite, une méthode de classe pour ajouter des produits à la liste, puis enfin, une méthode de classe « *calculer_prix* » sera disponible dans Stock afin de retourner le prix total de tous les produits.

Question 5

[solution n°7 p.25]

José souhaite maintenant pouvoir retirer un produit du stock. Créez une méthode *retirer_produit* dans la classe *Stock* qui prend en paramètre le nom du produit et le retire de la liste des produits si celui-ci existe.

B. Test

Exercice 1 : Quiz

[solution n°8 p.25]

Question 1

Pourquoi est-il déconseillé d'accéder directement à un attribut protégé (_attribut) depuis l'extérieur de la classe ?

- ○ Parce que cela peut entraîner une erreur de syntaxe
- ○ Parce que cela contourne le mécanisme d'encapsulation et peut entraîner des comportements inattendus
- ○ Parce que Python interdit l'accès à tout attribut protégé
- ○ Parce que cela supprime l'attribut

Question 2

Tous droits réservés à STUDI - Reproduction interdite

Quelle est la différence entre @property et une méthode normale pour accéder à un attribut ?

- @property rend un attribut accessible en tant que méthode sans avoir besoin d'appeler explicitement la méthode
- @property est plus rapide
- @property permet de créer une méthode statique
- @property cache l'attribut

Question 3

Que permet de faire la méthode __init__ ?

- Détruire un objet
- Initialiser un objet lors de sa création
- Supprimer les attributs d'une classe
- Copier une classe

Question 4

Comment contrôler la modification d'un attribut en Python ?

- En utilisant un constructeur
- En utilisant __init__
- En utilisant un setter via le décorateur @nom_attribut.setter
- En utilisant des méthodes statiques

Question 5

Quel est le but du mécanisme de name mangling en Python ?

- Empêcher l'accès direct aux attributs publics
- Renommer automatiquement les méthodes pour éviter les conflits
- Rendre les attributs privés moins accessibles depuis l'extérieur de la classe
- Faciliter la suppression d'attributs

Question 6

Comment Python renomme-t-il les attributs privés ?

- self.__attribut est renommé en self.__nom_attribut
- self.__attribut est renommé en _NomClasse__attribut
- self.__attribut est renommé en self.__attribut_privé
- self.__attribut est supprimé après sa création

Question 7

Tous droits réservés à STUDI - Reproduction interdite

Comment définissez-vous une classe en Python ?

- En utilisant def class NomClasse
- En utilisant class NomClasse:
- En utilisant new NomClasse
- En utilisant object NomClasse

Question 8

Que se passe-t-il si vous omettez d'utiliser self dans une méthode d'une classe ?

- Python renverra une erreur de syntaxe
- Python exécutera la méthode comme prévu
- Python ne pourra pas accéder aux attributs de l'objet
- Python supprimera l'attribut

Question 9

Que signifie instancier une classe ?

- Modifier une méthode
- Initialiser un objet à partir d'une classe
- Supprimer une classe
- Définir une fonction

Question 10

Comment appelle-t-on une méthode d'une classe sur un objet spécifique ?

- Classe.methode()
- objet.methode()
- methode()
- self.methode()

Solutions des exercices

Tous droits réservés à STUDI - Reproduction interdite

Exercice p. 8 Solution n°1

Question 1

Qu'est-ce qu'une classe en Python ?

- ○ Un type de boucle
- ◉ Un plan pour créer des objets
- ○ Une fonction spéciale
- ○ Une variable globale

Une classe est un modèle ou un plan qui permet de créer des objets avec des attributs et des méthodes définis.

Question 2

Quel est le rôle du constructeur __init__ dans une classe ?

- ◉ Il initialise les attributs de l'objet
- ○ Il supprime l'objet
- ○ Il ajoute l'objet à une liste
- ○ Il modifie une méthode

Le constructeur __init__ est utilisé pour initialiser les attributs de l'objet lorsqu'il est créé.

Question 3

Que représente self dans une méthode de classe ?

- ○ Une référence à la classe
- ◉ Une référence à l'objet actuel
- ○ Une variable globale
- ○ Un type de boucle

self est une référence à l'objet actuel de la classe, permettant d'accéder aux attributs et méthodes de l'instance courante.

Question 4

Quelle est la syntaxe correcte pour définir une classe en Python ?

- ○ class NomClasse {}
- ◉ class NomClasse:
- ○ def class NomClasse:
- ○ NomClasse = class {}

La syntaxe correcte pour définir une classe en Python utilise le mot-clé class suivi du nom de la classe et des deux points.

Tous droits réservés à STUDI - Reproduction interdite

Question 5

Que signifie instancier une classe ?

- ○ Supprimer une classe
- ◉ Créer un objet à partir d'une classe
- ○ Modifier un attribut de la classe
- ○ Exécuter une méthode de la classe

Q Instancier une classe signifie créer un objet (instance) à partir de cette classe.

Question 6

Que se passe-t-il si vous ne définissez pas de constructeur dans une classe ?

- ○ Python ne pourra pas créer d'objets
- ◉ Python créera un constructeur par défaut
- ○ L'objet aura des valeurs aléatoires
- ○ Python lancera une erreur

Q Si aucun constructeur __init__ n'est défini, Python en créera un par défaut qui permet de créer l'objet sans initialiser d'attributs spécifiques.

Question 7

Comment ajouter un attribut à un objet dans une classe ?

- ◉ Utiliser self.attribut
- ○ Utiliser class.attribut
- ○ Utiliser def attribut
- ○ Utiliser attr.attribut

Q Les attributs d'un objet sont définis à l'aide de self.attribut, où attribut est le nom de la propriété que l'on souhaite ajouter à l'objet.

Question 8

Comment accéder à un attribut d'instance à l'intérieur d'une méthode de classe ?

- ○ En utilisant class.attribut
- ◉ En utilisant self.attribut
- ○ En utilisant attribut.instance
- ○ En utilisant __init__(attribut)

Q À l'intérieur d'une méthode de classe, vous utilisez self pour accéder aux attributs de l'instance (objet).

Question 9

Que renvoie la fonction type() lorsqu'elle est appliquée à un objet ?

Tous droits réservés à STUDI - Reproduction interdite

- ○ Le nom de la méthode
- ◉ Le type de l'objet (la classe à laquelle il appartient)
- ○ La taille de l'objet
- ○ La valeur des attributs de l'objet

type(objet) renvoie le type de l'objet, c'est-à-dire la classe à laquelle il appartient.

Question 10

Que se passe-t-il si vous tentez d'accéder à un attribut inexistant d'un objet ?

- ○ Python renverra None
- ◉ Python lancera une exception AttributeError
- ○ Python créera automatiquement cet attribut
- ○ Python renverra une valeur par défaut

Si vous tentez d'accéder à un attribut inexistant d'un objet, Python lancera une exception AttributeError.

Exercice p. 13 **Solution n°2**

Question 1

Qu'est-ce que l'encapsulation en Python ?

- ○ La création d'objets
- ◉ La protection des attributs d'une classe
- ○ La suppression des méthodes inutiles
- ○ L'héritage de classes

L'encapsulation consiste à protéger les attributs d'une classe pour qu'ils ne puissent pas être modifiés directement depuis l'extérieur.

Question 2

Que signifie le préfixe d'un attribut avec un seul underscore (_attribut) en Python ?

- ○ L'attribut est public
- ○ L'attribut est privé et inaccessible
- ◉ L'attribut est protégé, mais accessible depuis l'extérieur de la classe
- ○ L'attribut est accessible uniquement dans les méthodes statiques

Un seul underscore indique que l'attribut est protégé. C'est une convention qui suggère que cet attribut ne devrait pas être modifié directement, bien qu'il soit techniquement accessible.

Question 3

Que signifie le double underscore (__attribut) en Python ?

Tous droits réservés à STUDI - Reproduction interdite

- ⦿ Il cache l'attribut pour qu'il ne soit pas accessible directement depuis l'extérieur
- ○ Il empêche l'attribut d'être modifié même depuis l'intérieur de la classe
- ○ Il crée un attribut partagé entre toutes les instances
- ○ Il rend l'attribut accessible uniquement dans une méthode spécifique

Q Le double underscore déclenche le mécanisme de **name mangling**, rendant l'attribut inaccessible directement depuis l'extérieur, protégeant ainsi son accès.

Question 4

Comment accède-t-on à un attribut privé défini avec double underscore (__attribut) ?

- ○ En utilisant self.__attribut
- ⦿ En utilisant _NomClasse__attribut
- ○ En utilisant @property
- ○ Il est impossible d'accéder à un attribut privé

Q En interne, Python modifie le nom des attributs préfixés par __, et vous pouvez y accéder via _NomClasse__attribut, bien que cela soit fortement déconseillé.

Question 5

Quel est l'avantage principal du décorateur @property ?

- ○ Il supprime un attribut
- ⦿ Il permet de définir un getter pour un attribut de manière sécurisée
- ○ Il permet d'accéder à des méthodes privées
- ○ Il empêche l'exécution de méthodes

Q Le décorateur @property permet de transformer une méthode en un attribut accessible comme une variable, tout en conservant un contrôle total sur l'accès et la modification de cet attribut.

Question 6

Que fait un setter dans une propriété Python ?

- ○ Il permet d'accéder à un attribut protégé
- ○ Il empêche la modification d'un attribut
- ⦿ Il permet de définir des règles lors de la modification d'un attribut
- ○ Il renvoie une erreur si l'attribut est modifié

Q Un setter vous permet de contrôler comment un attribut peut être modifié en définissant des règles ou des validations avant de modifier la valeur de cet attribut.

Question 7

Quelle est l'utilité du self dans les getters et setters ?

Tous droits réservés à STUDI - Reproduction interdite

- ○ Il empêche l'accès à l'attribut
- ◉ Il fait référence à l'objet courant, permettant d'accéder aux attributs de l'instance
- ○ Il modifie le type de l'attribut
- ○ Il remplace un attribut public par un privé

Q self fait référence à l'instance de la classe et permet d'accéder aux attributs et méthodes de cet objet.

Question 8

Pourquoi encapsuler les attributs en Python ?

- ○ Pour empêcher les utilisateurs de la classe d'accéder aux méthodes
- ◉ Pour sécuriser les données et les protéger d'un accès ou d'une modification non autorisés
- ○ Pour rendre les attributs inaccessibles même dans la classe
- ○ Pour éviter les erreurs de syntaxe

Q L'encapsulation permet de sécuriser les attributs et de s'assurer qu'ils ne sont pas modifiés de manière inappropriée depuis l'extérieur de la classe.

Question 9

Comment empêche-t-on la modification directe d'un attribut tout en permettant l'accès contrôlé ?

- ○ En définissant l'attribut avec un double underscore
- ○ En utilisant une méthode publique pour modifier l'attribut
- ◉ En utilisant @property pour accéder à l'attribut et un setter pour le modifier
- ○ En utilisant __del__

Q En utilisant @property pour accéder à l'attribut et un setter pour le modifier, vous pouvez contrôler l'accès et la modification des attributs de manière sécurisée.

Question 10

Quelle est la convention pour indiquer qu'un attribut est « *protégé* » mais pas totalement privé ?

- ○ Utiliser self.attribut
- ○ Utiliser self.__attribut
- ◉ Utiliser self._attribut
- ○ Utiliser self.attribut_protected

Q Le préfixe d'un seul underscore (_attribut) est la convention en Python pour indiquer qu'un attribut est protégé et ne devrait pas être accédé ou modifié directement.

Tous droits réservés à STUDI - Reproduction interdite

p. 16 Solution n°3

```
class Produit:
    def __init__(self, nom, prix, quantite_stock):
        self.nom = nom
        self._prix = prix
        self._quantite_stock = quantite_stock
```

p. 16 Solution n°4

```
@property
    def prix(self):
        return self._prix
    @prix.setter
    def prix(self, valeur):
        if valeur < 0:
            raise ValueError("Le prix ne peut pas être négatif.")
        self._prix = valeur
```

Explication :

- Le getter permet de récupérer la valeur de l'attribut _prix sans exposer directement la variable. Avec @property, nous transformons la méthode prix() en un attribut accessible comme une variable normale.
- Le setter contrôle et valide la valeur passée à l'attribut prix. Si la valeur est inférieure à 0, une exception est levée. Cela garantit que le prix reste toujours une valeur positive.

p. 16 Solution n°5

```
    @property
    def quantite_stock(self):
        return self._quantite_stock
    @quantite_stock.setter
    def quantite_stock(self, valeur):
        if valeur > 1000:
            raise ValueError("La quantité en stock ne peut pas dépasser 1000.")
        self._quantite_stock = valeur
```

Explication :

- Le getter pour quantite_stock permet d'accéder à l'attribut _quantite_stock de manière sécurisée.
- Le setter pour quantite_stock impose une limite maximale de 1000 unités en stock. Si la quantité proposée dépasse cette valeur, une exception est levée. Cela permet de contrôler la gestion du stock et d'éviter des valeurs anormales.

p. 16 Solution n°6

```
class Stock:
    def __init__(self):
        self.liste_produits = []

    def ajouter_produit(self, produit):
        """Ajoute un produit à la liste des produits en stock"""
        self.liste_produits.append(produit)

```


Tous droits réservés à STUDI - Reproduction interdite

```
    def calculer_prix_total(self):
        """Calcule la somme totale du prix des produits en stock"""
        prix_total = 0
        for produit in self.liste_produits:
            prix_total += produit.prix * produit.quantite_stock
        return prix_total
```

p. 16 Solution n°7

```
class Stock:
    def __init__(self):
        self.liste_produits = []

    def ajouter_produit(self, produit):
        """Ajoute un produit à la liste des produits en stock"""
        self.liste_produits.append(produit)

    def retirer_produit(self, nom_produit):
        """Retire un produit de la liste des produits en stock"""
        for produit in self.liste_produits:
            if produit.nom == nom_produit:
                self.liste_produits.remove(produit)
                print(f"Produit '{nom_produit}' retiré du stock.")
                return
        print(f"Produit '{nom_produit}' introuvable dans le stock.")

    def calculer_prix_total(self):
        """Calcule la somme totale du prix des produits en stock"""
        prix_total = 0
        for produit in self.liste_produits:
            prix_total += produit.prix * produit.quantite_stock
        return prix_total
```

Exercice p. 16 Solution n°8

Question 1

Pourquoi est-il déconseillé d'accéder directement à un attribut protégé (_attribut) depuis l'extérieur de la classe ?

- ○ Parce que cela peut entraîner une erreur de syntaxe
- ◉ Parce que cela contourne le mécanisme d'encapsulation et peut entraîner des comportements inattendus
- ○ Parce que Python interdit l'accès à tout attribut protégé
- ○ Parce que cela supprime l'attribut

Accéder directement à un attribut protégé contourne l'encapsulation et peut entraîner des modifications non souhaitées ou des comportements inattendus.

Question 2

Quelle est la différence entre @property et une méthode normale pour accéder à un attribut ?

- ◉ @property rend un attribut accessible en tant que méthode sans avoir besoin d'appeler explicitement la méthode

Tous droits réservés à STUDI - Reproduction interdite

- ○ @property est plus rapide
- ○ @property permet de créer une méthode statique
- ○ @property cache l'attribut

Q Le décorateur @property permet de définir une méthode qui peut être appelée comme un attribut, sans parenthèses, rendant l'accès plus naturel.

Question 3

Que permet de faire la méthode __init__ ?

- ○ Détruire un objet
- ⦿ Initialiser un objet lors de sa création
- ○ Supprimer les attributs d'une classe
- ○ Copier une classe

Q La méthode __init__ est un constructeur utilisé pour initialiser un objet lors de sa création. Elle permet de définir les attributs de l'objet.

Question 4

Comment contrôler la modification d'un attribut en Python ?

- ○ En utilisant un constructeur
- ○ En utilisant __init__
- ⦿ En utilisant un setter via le décorateur @nom_attribut.setter
- ○ En utilisant des méthodes statiques

Q Un setter permet de contrôler la modification d'un attribut en Python via le décorateur @nom_attribut.setter, offrant un contrôle sur la validation ou la transformation des données.

Question 5

Quel est le but du mécanisme de name mangling en Python ?

- ○ Empêcher l'accès direct aux attributs publics
- ○ Renommer automatiquement les méthodes pour éviter les conflits
- ⦿ Rendre les attributs privés moins accessibles depuis l'extérieur de la classe
- ○ Faciliter la suppression d'attributs

Q Le name mangling modifie le nom des attributs privés pour les rendre moins accessibles directement depuis l'extérieur de la classe.

Question 6

Comment Python renomme-t-il les attributs privés ?

Tous droits réservés à STUDI - Reproduction interdite

- ○ self.__attribut est renommé en self.__nom_attribut
- ◉ self.__attribut est renommé en _NomClasse__attribut
- ○ self.__attribut est renommé en self.__attribut_privé
- ○ self.__attribut est supprimé après sa création

Python renomme les attributs privés avec la syntaxe _NomClasse__attribut pour les rendre moins accessibles depuis l'extérieur de la classe.

Question 7

Comment définissez-vous une classe en Python ?

- ○ En utilisant def class NomClasse
- ◉ En utilisant class NomClasse:
- ○ En utilisant new NomClasse
- ○ En utilisant object NomClasse

En Python, une classe est définie avec le mot-clé class suivi du nom de la classe et d'un double point.

Question 8

Que se passe-t-il si vous omettez d'utiliser self dans une méthode d'une classe ?

- ○ Python renverra une erreur de syntaxe
- ○ Python exécutera la méthode comme prévu
- ◉ Python ne pourra pas accéder aux attributs de l'objet
- ○ Python supprimera l'attribut

Le mot-clé self fait référence à l'instance de l'objet et permet d'accéder à ses attributs et méthodes. Sans self, Python ne sait pas sur quel objet travailler.

Question 9

Que signifie instancier une classe ?

- ○ Modifier une méthode
- ◉ Initialiser un objet à partir d'une classe
- ○ Supprimer une classe
- ○ Définir une fonction

Instancier une classe signifie créer un objet à partir de cette classe.

Question 10

Comment appelle-t-on une méthode d'une classe sur un objet spécifique ?

Tous droits réservés à STUDI - Reproduction interdite

- ○ Classe.methode()
- ◉ objet.methode()
- ○ methode()
- ○ self.methode()

Pour appeler une méthode d'une classe sur un objet, vous utilisez objet.methode(), où objet est une instance de la classe.

Tous droits réservés à STUDI - Reproduction interdite